

1

- Creating and Running Python Programs
- Python’s “Beautiful Heart”

Rapid Introduction to Procedural Programming



This chapter provides enough information to get you started writing Python programs. We strongly recommend that you install Python if you have not already done so, so that you can get hands-on experience to reinforce what you learn here. (The Introduction explains how to obtain and install Python on all major platforms; 4 ◀.)

This chapter’s first section shows you how to create and execute Python programs. You can use your favorite plain text editor to write your Python code, but the IDLE programming environment discussed in this section provides not only a code editor, but also additional functionality, including facilities for experimenting with Python code, and for debugging Python programs.

The second section presents eight key pieces of Python that on their own are sufficient to write useful programs. These pieces are all covered fully in later chapters, and as the book progresses they are supplemented by all of the rest of Python so that by the end of the book, you will have covered the whole language and will be able to use all that it offers in your programs.

The chapter’s final section introduces two short programs which use the subset of Python features introduced in the second section so that you can get an immediate taste of Python programming.

Creating and Running Python Programs



Python code can be written using any plain text editor that can load and save text using either the ASCII or the UTF-8 Unicode character encoding. By default, Python files are assumed to use the UTF-8 character encoding, a superset of ASCII that can represent pretty well every character in every language. Python files normally have an extension of `.py`, although on some Unix-like sys-

Character
encodings

► 91

tems (e.g., Linux and Mac OS X) some Python applications have no extension, and Python GUI (Graphical User Interface) programs usually have an extension of `.pyw`, particularly on Windows and Mac OS X. In this book we always use an extension of `.py` for Python console programs and Python modules, and `.pyw` for GUI programs. All the examples presented in this book run unchanged on all platforms that have Python 3 available.

Just to make sure that everything is set up correctly, and to show the classical first example, create a file called `hello.py` in a plain text editor (Windows Notepad is fine—we'll use a better editor shortly), with the following contents:

```
#!/usr/bin/env python3

print("Hello", "World!")
```

The first line is a comment. In Python, comments begin with a `#` and continue to the end of the line. (We will explain the rather cryptic comment in a moment.) The second line is blank—outside quoted strings, Python ignores blank lines, but they are often useful to humans to break up large blocks of code to make them easier to read. The third line is Python code. Here, the `print()` function is called with two arguments, each of type `str` (string; i.e., a sequence of characters).

Each statement encountered in a `.py` file is executed in turn, starting with the first one and progressing line by line. This is different from some other languages, for example, C++ and Java, which have a particular function or method with a special name where they start from. The flow of control can of course be diverted as we will see when we discuss Python's control structures in the next section.

We will assume that Windows users keep their Python code in the `C:\py3eg` directory and that Unix (i.e., Unix, Linux, and Mac OS X) users keep their code in the `$HOME/py3eg` directory. Save `hello.py` into the `py3eg` directory and close the text editor.

Now that we have a program, we can run it. Python programs are executed by the Python interpreter, and normally this is done inside a console window. On Windows the console is called “Console”, or “DOS Prompt”, or “MS-DOS Prompt”, or something similar, and is usually available from Start→All Programs→Accessories. On Mac OS X the console is provided by the Terminal.app program (located in Applications/Utilities by default), available using Finder, and on other Unixes, we can use an `xterm` or the console provided by the windowing environment, for example, `konsole` or `gnome-terminal`.

Start up a console, and on Windows enter the following commands (which assume that Python is installed in the default location)—the console's output is shown in lightface; what you type is shown in **bold**:

```
C:\>cd c:\py3eg
C:\py3eg>c:\python31\python.exe hello.py
```

Since the `cd` (change directory) command has an absolute path, it doesn't matter which directory you start out from.

Unix users enter this instead (assuming that Python 3 is in the `PATH`):*

```
$ cd $HOME/py3eg
$ python3 hello.py
```

In both cases the output should be the same:

```
Hello World!
```

Note that unless stated otherwise, Python's behavior on Mac OS X is the same as that on any other Unix system. In fact, whenever we refer to "Unix" it can be taken to mean Linux, BSD, Mac OS X, and most other Unixes and Unix-like systems.

Although the program has just one executable statement, by running it we can infer some information about the `print()` function. For one thing, `print()` is a built-in part of the Python language—we didn't need to "import" or "include" it from a library to make use of it. Also, it separates each item it prints with a single space, and prints a newline after the last item is printed. These are default behaviors that can be changed, as we will see later. Another thing worth noting about `print()` is that it can take as many or as few arguments as we care to give it.

`print()`
► 181

Typing such command lines to invoke our Python programs would quickly become tedious. Fortunately, on both Windows and Unix we can use more convenient approaches. Assuming we are in the `py3eg` directory, on Windows we can simply type:

```
C:\py3eg>hello.py
```

Windows uses its registry of file associations to automatically call the Python interpreter when a filename with extension `.py` is entered in a console.

Unfortunately, this convenience does not always work, since some versions of Windows have a bug that sometimes affects the execution of interpreted programs that are invoked as the result of a file association. This isn't specific to Python; other interpreters and even some `.bat` files are affected by the bug too. If this problem arises, simply invoke Python directly rather than relying on the file association.

If the output on Windows is:

*The Unix prompt may well be different from the `$` shown here; it does not matter what it is.

```
('Hello', 'World!')
```

then it means that Python 2 is on the system and is being invoked instead of Python 3. One solution to this is to change the .py file association from Python 2 to Python 3. The other (less convenient, but safer) solution is to put the Python 3 interpreter in the path (assuming it is installed in the default location), and execute it explicitly each time. (This also gets around the Windows file association bug mentioned earlier.) For example:

```
C:\py3eg>path=c:\python31;%path%
C:\py3eg>python hello.py
```

It might be more convenient to create a py3.bat file with the single line `path=c:\python31;%path%` and to save this file in the `C:\Windows` directory. Then, whenever you start a console for running Python 3 programs, begin by executing `py3.bat`. Or alternatively you can have `py3.bat` executed automatically. To do this, change the console's properties (find the console in the Start menu, then right-click it to pop up its Properties dialog), and in the Shortcut tab's Target string, append the text " /u /k c:\windows\py3.bat" (note the space before, between, and after the "/u" and "/k" options, and be sure to add this at the end after "cmd.exe").

On Unix, we must first make the file executable, and then we can run it:

```
$ chmod +x hello.py
$ ./hello.py
```

We need to run the `chmod` command only once of course; after that we can simply enter `./hello.py` and the program will run.

On Unix, when a program is invoked in the console, the file's first two bytes are read.* If these bytes are the ASCII characters `#!`, the shell assumes that the file is to be executed by an interpreter and that the file's first line specifies which interpreter to use. This line is called the *shebang* (shell execute) line, and if present must be the first line in the file.

The shebang line is commonly written in one of two forms, either:

```
#!/usr/bin/python3
```

or:

```
#!/usr/bin/env python3
```

If written using the first form, the specified interpreter is used. This form may be necessary for Python programs that are to be run by a web server,

*The interaction between the user and the console is handled by a "shell" program. The distinction between the console and the shell does not concern us here, so we use the terms interchangeably.

although the specific path may be different from the one shown. If written using the second form, the first python3 interpreter found in the shell's current environment is used. The second form is more versatile because it allows for the possibility that the Python 3 interpreter is not located in /usr/bin (e.g., it could be in /usr/local/bin or installed under \$HOME). The shebang line is not needed (but is harmless) under Windows; all the examples in this book have a shebang line of the second form, although we won't show it.

Note that for Unix systems we assume that the name of Python 3's executable (or a soft link to it) in the PATH is python3. If this is not the case, you will need to change the shebang line in the examples to use the correct name (or correct name and path if you use the first form), or create a soft link from the Python 3 executable to the name python3 somewhere in the PATH.

Many powerful plain text editors, such as Vim and Emacs, come with built-in support for editing Python programs. This support typically involves providing color syntax highlighting and correctly indenting or unindenting lines. An alternative is to use the IDLE Python programming environment. On Windows and Mac OS X, IDLE is installed by default. On Unixes IDLE is built along with the Python interpreter if you build from the tarball, but if you use a package manager, IDLE is usually provided as a separate package as described in the Introduction.

As the screenshot in Figure 1.1 shows, IDLE has a rather retro look that harks back to the days of Motif on Unix and Windows 95. This is because it uses the Tk-based Tkinter GUI library (covered in Chapter 15) rather than one of the more powerful modern GUI libraries such as PyGtk, PyQt, or wxPython. The reasons for the use of Tkinter are a mixture of history, liberal license conditions, and the fact that Tkinter is much smaller than the other GUI libraries. On the plus side, IDLE comes as standard with Python and is very simple to learn and use.

IDLE provides three key facilities: the ability to enter Python expressions and code and to see the results directly in the Python Shell; a code editor that provides Python-specific color syntax highlighting and indentation support; and a debugger that can be used to step through code to help identify and kill bugs. The Python Shell is especially useful for trying out simple algorithms, snippets of code, and regular expressions, and can also be used as a very powerful and flexible calculator.

Several other Python development environments are available, but we recommend that you use IDLE, at least at first. An alternative is to create your programs in the plain text editor of your choice and debug using calls to `print()`.

It is possible to invoke the Python interpreter without specifying a Python program. If this is done the interpreter starts up in interactive mode. In this mode it is possible to enter Python statements and see the results exactly the same as when using IDLE's Python Shell window, and with the same `>>>`



```
Python Shell
File Edit Shell Debug Options Windows Help

>>> import SortedDict
>>> file_sizes = SortedDict.SortedDict(key=lambda x: x.lower())
>>> for name in os.listdir("."):
>>>     file_sizes[name] = os.path.getsize(name)

>>> len(file_sizes)
205
>>> print(file_sizes)
{'Abstract.py': 4591, 'Abstract.pyc': 8716, 'Account.py': 5354, 'Account.pyc': 7172, 'alltests.py': 58554, 'Appliance.py': 2909, 'Appliance.pyc': 3404, 'Ascii.py': 1668, 'Ascii.pyc': 1621, 'Atomic.py': 5262, 'Atomic.pyc': 4999, 'average1_ans.py': 1223, 'average2_ans.py': 1765, 'awfulpoetry1_ans.py': 1304, 'awfulpoetry2_ans.py': 1576, 'base64image.py': 1734, 'bigdigits.py': 1890, 'bigdigits_ans.py': 1961, 'BikeStock.py': 9516, 'BikeStock.pyc': 11649, 'BikeStock_ans.py': 9488, 'BikeStock_ans.pyc': 11744, 'BinaryRecordFile.py': 9191, 'BinaryRecordFile.pyc': 10410, 'BinaryRecordFile_ans.py': 5231, 'BinaryReco
```

Figure 1.1 IDLE's Python Shell

prompts. But IDLE is much easier to use, so we recommend using IDLE for experimenting with code snippets. The short interactive examples we show are all assumed to be entered in an interactive Python interpreter or in IDLE's Python Shell.

We now know how to create and run Python programs, but clearly we won't get very far knowing only a single function. In the next section we will considerably increase our Python knowledge. This will make us able to create short but useful Python programs, something we will do in this chapter's last section.

Python's "Beautiful Heart"

In this section we will learn about eight key pieces of Python, and in the next section we will show how these pieces can be used to write a couple of small but realistic programs. There is much more to say about all of the things covered in this section, so if as you read it you feel that Python is missing something or that things are sometimes done in a long-winded way, peek ahead using the forward references or using the table of contents or index, and you will almost certainly find that Python has the feature you want and often has more concise forms of expression than we show here—and a lot more besides.

Piece #1: Data Types

One fundamental thing that any programming language must be able to do is represent items of data. Python provides several built-in data types, but we will concern ourselves with only two of them for now. Python represents

integers (positive and negative whole numbers) using the `int` type, and it represents strings (sequences of Unicode characters) using the `str` type. Here are some examples of integer and string literals:

```
-973
210624583337114373395836055367340864637790190801098222508621955072
0
"Infinitely Demanding"
'Simon Critchley'
'positively αβγ€÷©'
''
```

Incidentally, the second number shown is 2^{217} —the size of Python's integers is limited only by machine memory, not by a fixed number of bytes. Strings can be delimited by double or single quotes, as long as the same kind are used at both ends, and since Python uses Unicode, strings are not limited to ASCII characters, as the penultimate string shows. An empty string is simply one with nothing between the delimiters.

Python uses square brackets (`[]`) to access an item from a sequence such as a string. For example, if we are in a Python Shell (either in the interactive interpreter, or in IDLE) we can enter the following—the Python Shell's output is shown in lightface; what you type is shown in **bold**:

```
>>> "Hard Times"[5]
'T'
>>> "giraffe"[0]
'g'
```

Traditionally, Python Shells use `>>>` as their prompt, although this can be changed. The square brackets syntax can be used with data items of any data type that is a sequence, such as strings and lists. This consistency of syntax is one of the reasons that Python is so beautiful. Note that all Python index positions start at 0.

In Python, both `str` and the basic numeric types such as `int` are *immutable*—that is, once set, their value cannot be changed. At first this appears to be a rather strange limitation, but Python's syntax means that this is a non-issue in practice. The only reason for mentioning it is that although we can use square brackets to retrieve the character at a given index position in a string, we cannot use them to set a new character. (Note that in Python a character is simply a string of length 1.)

To convert a data item from one type to another we can use the syntax `datatype(item)`. For example:

```
>>> int("45")
45
```

```
>>> str(912)
'912'
```

The `int()` conversion is tolerant of leading and trailing whitespace, so `int(" 45 ")` would have worked just as well. The `str()` conversion can be applied to almost any data item. We can easily make our own custom data types support `str()` conversion, and also `int()` or other conversions if they make sense, as we will see in Chapter 6. If a conversion fails, an exception is raised—we briefly introduce exception-handling in Piece #5, and fully cover exceptions in Chapter 4.

Strings and integers are fully covered in Chapter 2, along with other built-in data types and some data types from Python’s standard library. That chapter also covers operations that can be applied to immutable sequences, such as strings.

Piece #2: Object References



Once we have some data types, the next thing we need are variables in which to store them. Python doesn’t have variables as such, but instead has *object references*. When it comes to immutable objects like ints and strs, there is no discernable difference between a variable and an object reference. As for mutable objects, there is a difference, but it rarely matters in practice. We will use the terms *variable* and *object reference* interchangeably.

Shallow
and
deep
copying

► 146

Let’s look at a few tiny examples, and then discuss some of the details.

```
x = "blue"
y = "green"
z = x
```

The syntax is simply *objectReference* = *value*. There is no need for predeclaration and no need to specify the value’s type. When Python executes the first statement it creates a `str` object with the text “blue”, and creates an object reference called `x` that refers to the `str` object. For all practical purposes we can say that “variable `x` has been assigned the ‘blue’ string”. The second statement is similar. The third statement creates a new object reference called `z` and sets it to refer to the same object that the `x` object reference refers to (in this case the `str` containing the text “blue”).

The `=` operator is not the same as the variable assignment operator in some other languages. The `=` operator binds an object reference to an object in memory. If the object reference already exists, it is simply re-bound to refer to the object on the right of the `=` operator; if the object reference does not exist it is created by the `=` operator.

Let's continue with the `x, y, z` example, and do some rebinding—as noted earlier, comments begin with a `#` and continue until the end of the line:

```
print(x, y, z) # prints: blue green blue
z = y
print(x, y, z) # prints: blue green green
x = z
print(x, y, z) # prints: green green green
```

After the fourth statement (`x = z`), all three object references are referring to the same str. Since there are no more object references to the "blue" string, Python is free to garbage-collect it.

Figure 1.2 shows the relationship between objects and object references schematically.

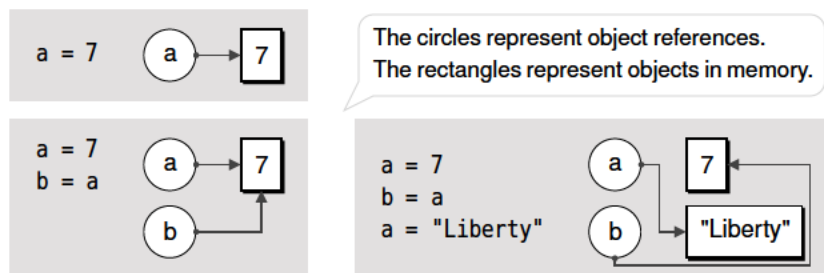


Figure 1.2 Object references and objects

The names used for object references (called *identifiers*) have a few restrictions. In particular, they may not be the same as any of Python's keywords, and must start with a letter or an underscore and be followed by zero or more nonwhite-space letter, underscore, or digit characters. There is no length limit, and the letters and digits are those defined by Unicode, that is, they include, but are not limited to, ASCII's letters and digits ("a", "b", ..., "z", "A", "B", ..., "Z", "0", "1", ..., "9"). Python identifiers are case-sensitive, so for example, `LIMIT`, `Limit`, and `limit` are three different identifiers. Further details and some slightly exotic examples are given in Chapter 2.

Python uses *dynamic typing*, which means that an object reference can be rebound to refer to a different object (which may be of a different data type) at any time. Languages that use strong typing (such as C++ and Java) allow only those operations that are defined for the data types involved to be performed. Python also applies this constraint, but it isn't called strong typing in Python's case because the valid operations can change—for example, if an object reference is re-bound to an object of a different data type. For example:

```
route = 866
print(route, type(route)) # prints: 866 <class 'int'>
```

```
route = "North"
print(route, type(route)) # prints: North <class 'str'>
```

Here we create a new object reference called `route` and set it to refer to a new `int` of value 866. At this point we could use `/` with `route` since division is a valid operation for integers. Then we reuse the `route` object reference to refer to a new `str` of value “North”, and the `int` object is scheduled for garbage collection since now no object reference refers to it. At this point using `/` with `route` would cause a `TypeError` to be raised since `/` is not a valid operation for a string.

The `type()` function returns the data type (also known as the “class”) of the data item it is given—this function can be very useful for testing and debugging, but would not normally appear in production code, since there is a better alternative as we will see in Chapter 6.

isin-
stance()
► 242

If we are experimenting with Python code inside the interactive interpreter or in a Python Shell such as the one provided by IDLE, simply typing the name of an object reference is enough to have Python print its value. For example:

```
>>> x = "blue"
>>> y = "green"
>>> z = x
>>> x
'blue'
>>> x, y, z
('blue', 'green', 'blue')
```

This is much more convenient than having to call the `print()` function all the time, but works only when using Python interactively—any programs and modules that we write must use `print()` or similar functions to produce output. Notice that Python displayed the last output in parentheses separated by commas—this signifies a tuple, that is, an ordered immutable sequence of objects. We will cover tuples in the next piece.

Piece #3: Collection Data Types



It is often convenient to hold entire collections of data items. Python provides several collection data types that can hold items, including associative arrays and sets. But here we will introduce just two: `tuple` and `list`. Python tuples and lists can be used to hold any number of data items of any data types. Tuples are *immutable*, so once they are created we cannot change them. Lists are *mutable*, so we can easily insert items and remove items whenever we want.

Tuples are created using commas `(,)`, as these examples show—and note that here, and from now on, we don’t use bold to distinguish what you type:

```
>>> "Denmark", "Finland", "Norway", "Sweden"
('Denmark', 'Finland', 'Norway', 'Sweden')
```

```
>>> "one",  
('one',)
```

When Python outputs a tuple it encloses it in parentheses. Many programmers emulate this and always enclose the tuple literals they write in parentheses. If we have a one-item tuple and want to use parentheses, we must still use the comma—for example, (1,). An empty tuple is created by using empty parentheses, (). The comma is also used to separate arguments in function calls, so if we want to pass a tuple literal as an argument we must enclose it in parentheses to avoid confusion.

Here are some example lists:

```
[1, 4, 9, 16, 25, 36, 49]  
['alpha', 'bravo', 'charlie', 'delta', 'echo']  
['zebra', 49, -879, 'armadillo', 200]  
[]
```

One way to create a list is to use square brackets ([]) as we have done here; later on we will see other ways. The fourth list shown is an empty list.

Under the hood, lists and tuples don't store data items at all, but rather object references. When lists and tuples are created (and when items are inserted in the case of lists), they take copies of the object references they are given. In the case of literal items such as integers or strings, an object of the appropriate data type is created in memory and suitably initialized, and then an object reference referring to the object is created, and it is this object reference that is put in the list or tuple.

Like everything else in Python, collection data types are objects, so we can nest collection data types inside other collection data types, for example, to create lists of lists, without formality. In some situations the fact that lists, tuples, and most of Python's other collection data types hold object references rather than objects makes a difference—this is covered in Chapter 3.

In procedural programming we call functions and often pass in data items as arguments. For example, we have already seen the `print()` function. Another frequently used Python function is `len()`, which takes a single data item as its argument and returns the “length” of the item as an `int`. Here are a few calls to `len()`:

```
>>> len(("one",))  
1  
>>> len([3, 5, 1, 2, "pause", 5])  
6  
>>> len("automatically")  
13
```

tuple
type

► 108

Creating
and
calling
func-
tions

► 36

list
type

► 113

Shallow
and
deep
copying

► 146

Tuples, lists, and strings are “sized”, that is, they are data types that have a notion of size, and data items of any such data type can be meaningfully passed to the `len()` function. (An exception is raised if a nonsized data item is passed to `len()`.)

Sized
► 383

All Python data items are *objects* (also called *instances*) of a particular data type (also called a *class*). We will use the terms *data type* and *class* interchangeably. One key difference between an object, and the plain items of data that some other languages provide (e.g., C++ or Java’s built-in numeric types), is that an object can have *methods*. Essentially, a method is simply a function that is called for a particular object. For example, the list type has an `append()` method, so we can append an object to a list like this:

```
>>> x = ["zebra", 49, -879, "aardvark", 200]
>>> x.append("more")
>>> x
['zebra', 49, -879, 'aardvark', 200, 'more']
```

The `x` object knows that it is a list (all Python objects know what their own data type is), so we don’t need to specify the data type explicitly. In the implementation of the `append()` method the first argument will be the `x` object itself—this is done automatically by Python as part of its syntactic support for methods.

The `append()` method mutates, that is, changes, the original list. This is possible because lists are mutable. It is also potentially more efficient than creating a new list with the original items and the extra item and then rebinding the object reference to the new list, particularly for very long lists.

In a procedural language the same thing could be achieved by using the list’s `append()` like this (which is perfectly valid Python syntax):

```
>>> list.append(x, "extra")
>>> x
['zebra', 49, -879, 'aardvark', 200, 'more', 'extra']
```

Here we specify the data type and the data type’s method, and give as the first argument the data item of the data type we want to call the method on, followed by any additional arguments. (In the face of inheritance there is a subtle semantic difference between the two syntaxes; the first form is the one that is most commonly used in practice. Inheritance is covered in Chapter 6.)

If you are unfamiliar with object-oriented programming this may seem a bit strange at first. For now, just accept that Python has conventional functions called like this: `functionName(arguments)`; and methods which are called like this: `objectName.methodName(arguments)`. (Object-oriented programming is covered in Chapter 6.)